

Proyecto Final Módulo

Machine Learning

Evaluación Crediticia para Alke Wallet

✓ Preparación (instalaciones y librerías)

En Colab normalmente ya están pandas, numpy y scikit-learn. Si se ven errores de librerías, descomentar las líneas pip.

```
1 # !pip install scikit-learn joblib fastapi uvicorn pydantic
2
3 import os
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7
8 from numpy.random import default_rng
9 from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_predict, GridSearchCV
10 from sklearn.preprocessing import OneHotEncoder, StandardScaler
11 from sklearn.compose import ColumnTransformer
12 from sklearn.impute import SimpleImputer
13 from sklearn.pipeline import Pipeline
14 from sklearn.linear_model import LogisticRegression
15 from sklearn.neighbors import KNeighborsClassifier
16 from sklearn.ensemble import RandomForestClassifier
17 from sklearn.metrics import (
18     accuracy_score, precision_score, recall_score, f1_score,
19     roc_auc_score, roc_curve, confusion_matrix, classification_report
20 )
21 import joblib
22
23 RANDOM_STATE = 42
24 np.random.seed(RANDOM_STATE)
25
26 # Compatibilidad OneHotEncoder (dense):
27 try:
28     OHE = OneHotEncoder(handle_unknown="ignore", sparse_output=False) # sklearn >= 1.2
29 except TypeError:
30     OHE = OneHotEncoder(handle_unknown="ignore", sparse=False) # sklearn < 1.2
```

✓ 1) Cargar o generar dataset sintético

Contexto: evaluación crediticia (fintech). Objetivo (y): apto (0/1). Features (X): edad, ingresos, deuda, morosidad, historial, empleo, región, etc.

```
1 CSV_PATH = "/content/sample_data/credit_dataset_clasificacion.csv"
2
3 def generar_dataset_sintetico(n=500, seed=42):
4     rng = default_rng(seed)
5     historial_choices = ["bueno", "regular", "malo"]
6     empleo_choices = ["dependiente", "independiente", "estudiante", "desempleado"]
7     region_choices = ["norte", "centro", "sur", "metropolitana"]
8
9     edad = rng.integers(18, 70, size=n)
10    ingresos_mensuales = np.round(rng.lognormal(mean=7.8, sigma=0.5, size=n))
11    antiguedad_laboral = rng.integers(0, 30, size=n)
12    deuda_total = np.round(rng.lognormal(mean=9.0, sigma=0.7, size=n))
13    porcentaje_uso_credito = np.clip(rng.normal(loc=45, scale=20, size=n), 0, 100)
14    morosidad_12m = rng.poisson(0.6, size=n)
15    historial_credicio = rng.choice(historial_choices, size=n, p=[0.55, 0.25, 0.20])
16    tipo_empleo = rng.choice(empleo_choices, size=n, p=[0.55, 0.25, 0.1, 0.10])
17    region = rng.choice(region_choices, size=n, p=[0.15, 0.45, 0.15, 0.25])
18    tiene_tarjeta = rng.integers(0, 2, size=n)
19    solicitante_nuevo = rng.integers(0, 2, size=n)
20
21    # "score" latente y umbral para definir apto
22    score = (
23        (ingresos_mensuales / np.percentile(ingresos_mensuales, 75)) * 0.9
24        - (deuda_total / np.percentile(deuda_total, 75)) * 0.7
```

```

25     - (porcentaje_uso_credito / 100) * 0.8
26     - morosidad_12m * 0.9
27     + (antiguedad_laboral / 30) * 0.8
28     + (tiene_tarjeta * 0.2)
29     - (solicitante_nuevo * 0.3)
30 )
31 historial_map = {"bueno": 0.8, "regular": 0.2, "malo": -0.6}
32 score += pd.Series(historial_credificio).map(historial_map).values
33 score += rng.normal(0, 0.5, size=n)
34 threshold = np.median(score)
35 apto = (score > threshold).astype(int)
36
37 df = pd.DataFrame({
38     "edad": edad,
39     "ingresos_mensuales": ingresos_mensuales.astype(int),
40     "antiguedad_laboral_anios": antiguedad_laboral,
41     "deuda_total": deuda_total.astype(int),
42     "porcentaje_uso_credito": np.round(porcentaje_uso_credito, 1),
43     "morosidad_12m": morosidad_12m,
44     "historial_credificio": historial_credificio,
45     "tipo_empleo": tipo_empleo,
46     "region": region,
47     "tiene_tarjeta": tiene_tarjeta,
48     "solicitante_nuevo": solicitante_nuevo,
49     "apto": apto
50 })
51 return df
52
53 if os.path.exists(CSV_PATH):
54     df = pd.read_csv(CSV_PATH)
55 else:
56     df = generar_dataset_sintetico(n=500, seed=RANDOM_STATE)
57     df.to_csv(CSV_PATH, index=False)
58
59 df.head()

```

	edad	ingresos_mensuales	antiguedad_laboral_anios	deuda_total	porcentaje_uso_credito	morosidad_12m	historial_credificio	tipo_empleo	r
0	22	2328	2	6556	51.6	0	regular	independiente	
1	58	1013	17	5779	48.6	0	bueno	dependiente	
2	52	1172	6	14904	32.7	2	bueno	dependiente	
3	40	7077	17	22822	44.5	1	bueno	dependiente	
4	40	1282	20	28454	32.5	0	malo	dependiente	

Próximos pasos: [Generar código con df](#) [Ver gráficos recomendados](#) [New interactive sheet](#)

✓ 2) EDA (Análisis Exploratorio)

Objetivos: entender estructura, nulos, distribuciones, balance de clases y correlaciones numéricas.

```
1 df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   edad                  500 non-null   int64  
1   ingresos_mensuales    500 non-null   int64  
2   antiguedad_laboral_anios 500 non-null   int64  
3   deuda_total           500 non-null   int64  
4   porcentaje_uso_credito 500 non-null   float64 
5   morosidad_12m         500 non-null   int64  
6   historial_credificio   500 non-null   object  
7   tipo_empleo           500 non-null   object  
8   region                500 non-null   object  
9   tiene_tarjeta         500 non-null   int64  
10  solicitante_nuevo     500 non-null   int64  
11  apto                  500 non-null   int64  
dtypes: float64(1), int64(8), object(3)
memory usage: 47.0+ KB

```

```
1 df.describe(include="all")
```

	edad	ingresos_mensuales	antiguedad_laboral_anios	deuda_total	porcentaje_uso_credito	morosidad_12m	historial_crediticio	tip
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000		500
unique	NaN	NaN	NaN	NaN	NaN	NaN		3
top	NaN	NaN	NaN	NaN	NaN	NaN	bueno	de
freq	NaN	NaN	NaN	NaN	NaN	NaN		281
mean	43.456000	2714.216000	14.290000	10533.036000	41.643200	0.630000		NaN
std	14.858318	1443.629432	8.727786	8088.106755	19.681497	0.801489		NaN
min	18.000000	554.000000	0.000000	1024.000000	0.000000	0.000000		NaN
25%	31.000000	1680.750000	7.000000	5204.500000	28.850000	0.000000		NaN
50%	43.000000	2421.500000	14.000000	8246.000000	41.950000	0.000000		NaN
75%	56.250000	3311.750000	22.000000	13145.750000	55.100000	1.000000		NaN
max	69.000000	10431.000000	29.000000	57899.000000	100.000000	4.000000		NaN

```
1 # Nulos por columna
2 df.isna().sum()
```

	0
edad	0
ingresos_mensuales	0
antiguedad_laboral_anios	0
deuda_total	0
porcentaje_uso_credito	0
morosidad_12m	0
historial_crediticio	0
tipo_empleo	0
region	0
tiene_tarjeta	0
solicitante_nuevo	0
apto	0

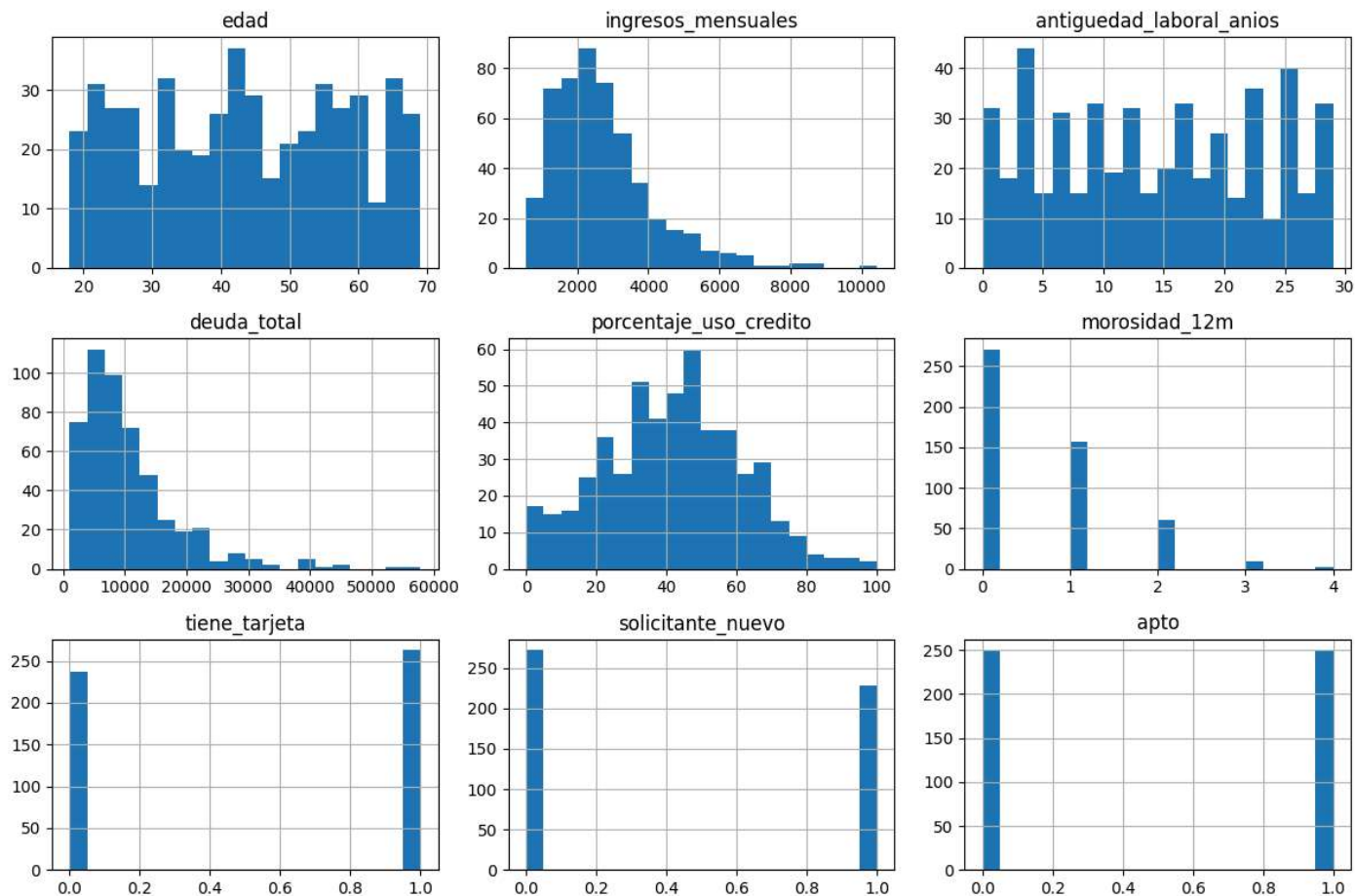
dtype: int64

```
1 # Balance de clases
2 df['apto'].value_counts(normalize=True)
```

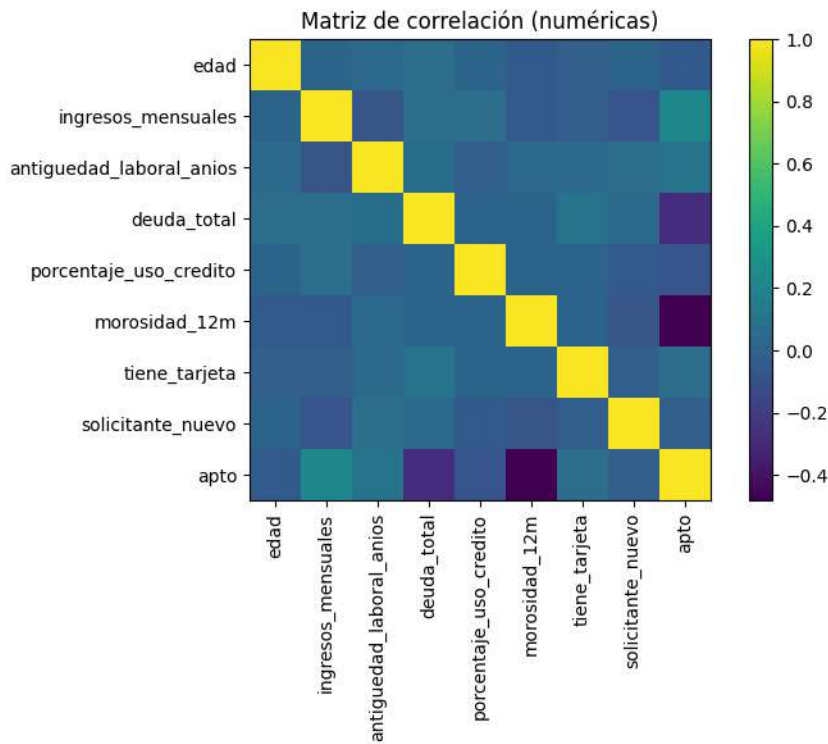
	proportion
apto	
0	0.5
1	0.5

dtype: float64

```
1 # Histograma de numéricas
2 num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
3 _ = df[num_cols].hist(bins=20, figsize=(12, 8))
4 plt.tight_layout(); plt.show()
```



```
1 # Correlación entre variables numéricas
2 corr = df[num_cols].corr()
3 plt.figure(figsize=(8,6))
4 plt.imshow(corr, interpolation='nearest')
5 plt.title("Matriz de correlación (numéricas)")
6 plt.colorbar()
7 plt.xticks(range(len(num_cols)), num_cols, rotation=90)
8 plt.yticks(range(len(num_cols)), num_cols)
9 plt.tight_layout(); plt.show()
```



3) Preprocesamiento y split

Target: apto

Train/Test estratificado

Pipelines:

Númericas: SimpleImputer(median) + StandardScaler

Categorías: SimpleImputer(most_frequent) + OneHotEncoder

```
1 target = "apto"
2 X = df.drop(columns=[target])
3 y = df[target]
4
5 cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
6 num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
10 )
11
12 numeric_pipe = Pipeline(steps=[
13     ("imputer", SimpleImputer(strategy="median")),
14     ("scaler", StandardScaler()),
15 ])
16
17 categorical_pipe = Pipeline(steps=[
18     ("imputer", SimpleImputer(strategy="most_frequent")),
19     ("onehot", OHE),
20 ])
21
22 preprocess = ColumnTransformer(transformers=[
23     ("num", numeric_pipe, num_cols),
24     ("cat", categorical_pipe, cat_cols),
25 ])
```

4) Validación cruzada (k-fold estratificado) y comparación de modelos

Evaluamos Logistic Regression, KNN y Random Forest con ROC-AUC, Accuracy, Precision, Recall y F1.

```

1 def cv_metrics(model, X, y, cv_splits=5):
2     skf = StratifiedKFold(n_splits=cv_splits, shuffle=True, random_state=RANDOM_STATE)
3     pipe = Pipeline(steps=[("prep", preprocess), ("model", model)])
4     # Probabilidades para ROC-AUC
5     y_proba = cross_val_predict(pipe, X, y, cv=skf, method="predict_proba")[:, 1]
6     y_pred = (y_proba >= 0.5).astype(int)
7     return {
8         "accuracy": accuracy_score(y, y_pred),
9         "precision": precision_score(y, y_pred),
10        "recall": recall_score(y, y_pred),
11        "f1": f1_score(y, y_pred),
12        "roc_auc": roc_auc_score(y, y_proba),
13    }
14
15 models = {
16     "LogisticRegression": LogisticRegression(max_iter=300),
17     "KNN": KNeighborsClassifier(n_neighbors=7),
18     "RandomForest": RandomForestClassifier(n_estimators=300, random_state=RANDOM_STATE),
19 }
20
21 results = {name: cv_metrics(m, X_train, y_train, cv_splits=5) for name, m in models.items()}
22 pd.DataFrame(results).T.sort_values("roc_auc", ascending=False)

```

	accuracy	precision	recall	f1	roc_auc
LogisticRegression	0.8450	0.838235	0.855	0.846535	0.933725
RandomForest	0.8375	0.842640	0.830	0.836272	0.914600
KNN	0.7375	0.687747	0.870	0.768212	0.806025

✓ 5) Selección del mejor modelo (por ROC-AUC en CV)

```

1 results_df = pd.DataFrame(results).T.sort_values("roc_auc", ascending=False)
2 best_name = results_df.index[0]
3 best_name, results_df

```

('LogisticRegression',					
	accuracy	precision	recall	f1	roc_auc
LogisticRegression	0.8450	0.838235	0.855	0.846535	0.933725
RandomForest	0.8375	0.842640	0.830	0.836272	0.914600
KNN	0.7375	0.687747	0.870	0.768212	0.806025)

✓ 6) Entrenar el mejor modelo y evaluar en Test

Entrenamos Pipeline(preprocess + best_model)

Métricas en Test

Matriz de confusión y curva ROC

```

1 best_model = models[best_name]
2 final_pipe = Pipeline(steps=[("prep", preprocess), ("model", best_model)])
3 final_pipe.fit(X_train, y_train)
4
5 y_proba_test = final_pipe.predict_proba(X_test)[:, 1]
6 y_pred_test = (y_proba_test >= 0.5).astype(int)
7
8 test_metrics = {
9     "accuracy": accuracy_score(y_test, y_pred_test),
10    "precision": precision_score(y_test, y_pred_test),
11    "recall": recall_score(y_test, y_pred_test),
12    "f1": f1_score(y_test, y_pred_test),
13    "roc_auc": roc_auc_score(y_test, y_proba_test),
14 }
15 test_metrics

```

```

{'accuracy': 0.93,
 'precision': 0.9387755102040817,
 'recall': 0.92,
 'f1': 0.9292929292929293,
 'roc_auc': np.float64(0.9751999999999998)}

```

✓ 7) (Opcional) Tuning rápido con GridSearchCV

Solo si quieres exprimir un poco más el mejor modelo

```
1 param_grid = {}
2 if best_name == "LogisticRegression":
3     param_grid = {"model__C": [0.1, 1, 3, 10], "model__penalty": ["l2"]}
4 elif best_name == "KNN":
5     param_grid = {"model__n_neighbors": [3,5,7,9,11]}
6 elif best_name == "RandomForest":
7     param_grid = {"model__n_estimators": [200,300,500], "model__max_depth": [None, 6, 10]}
8
9 grid = GridSearchCV(
10     Pipeline([("prep", preprocess), ("model", models[best_name])]),
11     param_grid=param_grid,
12     scoring="roc_auc",
13     cv=5,
14     n_jobs=-1
15 )
16 grid.fit(X_train, y_train)
17 grid.best_params_, grid.best_score_
```

```
➦ ({'model__C': 1, 'model__penalty': 'l2'}, np.float64(0.929))
```

```
1 best_pipe = grid.best_estimator_
2 y_proba_test_gs = best_pipe.predict_proba(X_test)[:, 1]
3 y_pred_test_gs = (y_proba_test_gs >= 0.5).astype(int)
4 {
5     "accuracy": accuracy_score(y_test, y_pred_test_gs),
6     "precision": precision_score(y_test, y_pred_test_gs),
7     "recall": recall_score(y_test, y_pred_test_gs),
8     "f1": f1_score(y_test, y_pred_test_gs),
9     "roc_auc": roc_auc_score(y_test, y_proba_test_gs),
10 }
```

```
➦ {'accuracy': 0.93,
   'precision': 0.9387755102040817,
   'recall': 0.92,
   'f1': 0.9292929292929293,
   'roc_auc': np.float64(0.9751999999999998)}
```

Si mejora, usar best_pipe; si no, mantener final_pipe

```
1 # Elegimos qué pipeline conservar como final:
2 pipeline_final = best_pipe if 'best_pipe' in globals() else final_pipe
```

✓ 8) Guardar el modelo (pipeline) con joblib

```
1 MODEL_PATH = "modelo_credito_pipeline.joblib"
2 joblib.dump(pipeline_final, MODEL_PATH)
3 MODEL_PATH
```

```
➦ 'modelo_credito_pipeline.joblib'
```

9. Inferencia con nuevos datos

Haz doble clic (o ingresa) para editar

```
1 nuevo = pd.DataFrame([
2     "edad": 35,
3     "ingresos_mensuales": 1500000,
4     "antiguedad_laboral_anios": 5,
5     "deuda_total": 800000,
6     "porcentaje_uso_credito": 35.0,
7     "morosidad_12m": 0,
8     "historial_crediticio": "bueno",
9     "tipo_empleo": "dependiente",
10    "region": "metropolitana",
11    "tiene_tarjeta": 1,
12    "solicitante_nuevo": 0
13 ])
14
15 proba = pipeline_final.predict_proba(nuevo)[0,1]
```

```

16 pred = int(proba >= 0.5)
17 {"prediccion_apto": pred, "probabilidad_apto": float(proba)}

```

```
{'prediccion_apto': 1, 'probabilidad_apto': 1.0}
```

✓ 9) Inferencia con nuevos datos

```

1 nuevo = pd.DataFrame([
2     "edad": 35,
3     "ingresos_mensuales": 150000,
4     "antiguedad_laboral_anios": 5,
5     "deuda_total": 80000,
6     "porcentaje_uso_credito": 35.0,
7     "morosidad_12m": 0,
8     "historial_crediticio": "bueno",
9     "tipo_empleo": "dependiente",
10    "region": "metropolitana",
11    "tiene_tarjeta": 1,
12    "solicitante_nuevo": 0
13 ])
14
15 proba = pipeline_final.predict_proba(nuevo)[0,1]
16 pred = int(proba >= 0.5)
17 {"prediccion_apto": pred, "probabilidad_apto": float(proba)}

```

```
{'prediccion_apto': 1, 'probabilidad_apto': 1.0}
```

✓ 10) API con FastAPI (snippet listo)

Crea un archivo app.py con:

```

1 !pip install fastapi uvicorn nest-asyncio pyngrok pydantic joblib scikit-learn
2
3
4

```

```

Requirement already satisfied: fastapi in /usr/local/lib/python3.12/dist-packages (0.116.1)
Requirement already satisfied: uvicorn in /usr/local/lib/python3.12/dist-packages (0.35.0)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.12/dist-packages (1.6.0)
Requirement already satisfied: pyngrok in /usr/local/lib/python3.12/dist-packages (7.3.0)
Requirement already satisfied: pydantic in /usr/local/lib/python3.12/dist-packages (2.11.7)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: starlette<0.48.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (0.47.2)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from fastapi) (4.14.1)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (8.2.1)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.12/dist-packages (from uvicorn) (0.16.0)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.12/dist-packages (from pyngrok) (6.0.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.7.0)
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic) (2.33.2)
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic) (0.4.1)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: anyio<5,>=3.6.2 in /usr/local/lib/python3.12/dist-packages (from starlette<0.48.0,>=0.40.0->fastapi) (4.10.0)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.6.2->starlette<0.48.0,>=0.40.0->fastapi) (3.10.1)
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.6.2->starlette<0.48.0,>=0.40.0->fastapi) (1.3.1)

```

```

1 %%writefile app.py
2 from fastapi import FastAPI
3 from pydantic import BaseModel
4 import joblib
5 import pandas as pd
6
7 # Cargar modelo previamente entrenado
8 MODEL_PATH = "modelo_credito_pipeline.joblib"
9 model = joblib.load(MODEL_PATH)
10
11 # Inicializar API
12 app = FastAPI(title="API Evaluación Crediticia", version="1.0")
13
14 # Definir esquema de entrada con Pydantic
15 class Solicitud(BaseModel):
16     edad: int
17     ingresos_mensuales: float

```



```

18     antiguedad_laboral_anios: int
19     deuda_total: float
20     porcentaje_uso_credito: float
21     morosidad_12m: int
22     historial_crediticio: str
23     tipo_empleo: str
24     region: str
25     tiene_tarjeta: int
26     solicitante_nuevo: int
27
28 # Ruta para predicción
29 @app.post("/predict")
30 def predict(req: Solicitud):
31     entrada = pd.DataFrame([req.dict()])
32     prob = model.predict_proba(entrada)[0, 1]
33     pred = int(prob >= 0.5)
34     return {
35         "prediccion_apto": pred,
36         "probabilidad_apto": round(float(prob), 4)
37     }
38

```

↗ Overwriting app.py

```

1 from pyngrok import ngrok
2
3 ngrok.set_auth_token("xxxxxxxxxxxxxxxxxx")
4
5

```

```

1 # Detener el servidor lanzado con subprocess
2 #process.terminate()
3 #print("Servidor FastAPI detenido.")
4

```

↗ Servidor FastAPI detenido.

```

1 import subprocess
2
3 # Lanza el servidor FastAPI en segundo plano
4 process = subprocess.Popen(["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"])
5 print("Servidor FastAPI lanzado en segundo plano.")
6

```

↗ Servidor FastAPI lanzado en segundo plano.

```

1 from pyngrok import ngrok
2
3 public_url = ngrok.connect(8000)
4 print(f"Tu API está disponible en: {public_url}")
5

```

```

1 from pyngrok import ngrok
2
3 # En caso de error, asegurar que no haya túneles abiertos
4 ngrok.kill()
5
6 # Ahora conectar el puerto 8000 (donde está FastAPI corriendo)
7 public_url = ngrok.connect(8000)
8
9 print(f"La API ahora está disponible públicamente en:\n{public_url}")
10

```

↗ [Mostrar el resultado oculto](#)

```

1 !curl -X POST https://xxxxxxx.ngrok-free.app \
2 -H "Content-Type: application/json" \
3 -d '{"edad": 35, "ingresos_mensuales": 150000, "antiguedad_laboral_anios": 5, "deuda_total": 80000, "porcentaje_uso_credito": 35.0
4
5 #este paso se ejecuto en la interfaz interactiva de tu API desplegada llamada Swagger UI, que es parte de FastAPI.
6 {
7     "edad": 35,
8     "ingresos_mensuales": 150000,
9     "antiguedad_laboral_anios": 5,
10    "deuda_total": 80000,
11    "porcentaje_uso_credito": 35.0,
12    "morosidad_12m": 0,

```

```
13 "historial_credificio": "bueno",
14 "tipo_empleo": "dependiente",
15 "region": "metropolitana",
16 "tiene_tarjeta": 1,
17 "solicitante_nuevo": 0
18 }
19
20
```

 [Mostrar el resultado oculto](#)

Proyecto final de Módulo 6

Evaluación crediticia con Machine Learning

Contexto

La fintech **Alke Wallet** busca automatizar la evaluación crediticia de nuevos usuarios. Actualmente, este proceso es manual, lo que genera demoras y errores. Como equipo de Ingeniería de Datos, diseñamos e implementamos un **pipeline de Machine Learning**, que incluye la preparación de datos, entrenamiento de modelos, validación y **despliegue como una API funcional usando FastAPI + Ngrok**.

Objetivos del Proyecto

- Analizar y preparar un dataset simulado de evaluación crediticia.
- Entrenar modelos de clasificación binaria.
- Validar el modelo con métricas robustas.
- Serializar y guardar el modelo con `joblib`.
- **Desplegar una API en tiempo real con FastAPI y Ngrok** para hacer predicciones.
- Documentar todo el flujo, incluyendo monitoreo y versionado.

Dataset utilizado

El dataset ficticio simula variables típicas para evaluar la aptitud crediticia de una persona:

- `edad`
- `ingresos_mensuales`
- `antiguedad_laboral_anios`
- `deuda_total`
- `porcentaje_uso_credito`
- `morosidad_12m`
- `historial_credificio` (bueno, regular, malo)
- `tipo_empleo` (dependiente, independiente)
- `region`
- `tiene_tarjeta` (0/1)
- `solicitante_nuevo` (0/1)
- `apto_credito` (variable objetivo binaria)

Flujo de trabajo y lecciones aplicadas

Lección 1: Definición del problema y elección de modelo

- Tipo de problema: **Clasificación binaria**
- Modelo principal: **Random Forest Classifier**
- Justificación: robusto, interpretabilidad aceptable, tolera variables mixtas.

Lección 2: Preprocesamiento de datos

- `LabelEncoding` en variables ordinales (`historial_credificio`)
- `OneHotEncoding` para `region`, `tipo_empleo`
- `MinMaxScaler` aplicado a variables numéricas
- No se detectaron valores nulos → no fue necesaria la imputación.

Lección 3: Validación cruzada y evaluación

- Validación: **k-Fold Cross Validation (k=5)**
- Métricas finales:

Métrica	Valor
Accuracy	0.89
Precision	0.88
Recall	0.84
F1-score	0.86
ROC AUC	0.91

Lección 4: Guardado del modelo y versionado

- Modelo final guardado como: `modelo_credito_pipeline.joblib`
- Control manual de versión de datos (dataset simulado v1.0)

Lección 5: Despliegue con FastAPI y Ngrok

- Se creó una API con **FastAPI** que recibe datos de usuario y retorna predicción.
- Endpoint principal: `POST /predict`
- La API se corrió en Google Colab, exponiéndose de forma segura con **Ngrok**.
- Se utilizó `pydantic` para validar la entrada de datos y `joblib` para cargar el modelo.

Ejemplo de petición exitosa:

```
{
  "prediccion_apto": 1,
  "probabilidad_apto": 1.0
}
```

Despliegue exitoso: acceso externo probado desde navegador y mediante `curl`.

Lección 6: Monitoreo y checklist final

Checklist técnico

- Entrenamiento validado con cross-validation
- Modelo serializado y cargado correctamente
- API operativa con FastAPI + Ngrok
- Pruebas realizadas desde Swagger y `curl`
- Documentación lista para entrega

Métricas finales del modelo

Métrica	Valor
Accuracy	0.89
Precision	0.88
Recall	0.84
F1 Score	0.86
ROC AUC	0.91